Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-213БВ-26

Студент:Лубонго Тереза Лузия

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 23.10.26

Москва, 2026

Постановка задачи

Вариант15.

Родительский процесс создает дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на запись. Родительский и дочерний процесс представлены разными программами(parent.exe и child.exe). Общий метод и алгоритм

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1, который связан со стандартным входным потоком дочернего процесса. Процесс child проверяет строки на валидность по правилу. Если строка соответствует правилу, то она выводится в стандартный поток вывода дочернего процесса (перенаправленный в файл). Иначе в pipe2 выводится информация об ошибке. Родительский процесс полученные от child ошибки выводит в свой стандартный поток вывода**.** строка должна начинаться с заглавной буквы.

решения

Общий метод и алгоритм решения

В данной лабораторной работе использовались следующие системные вызовы операционной системы Windows:

CreateFile() – создание или открытие файла для записи. Используется родительским процессом для создания файла, в который будут записываться валидные строки.

CreatePipe() – создание безымянного канала (pipe) для передачи данных между процессами. Создано два канала: pipe1 (для передачи строк от родителя к ребёнку) и pipe2 (для передачи сообщений об ошибках от ребёнка к родителю).

CreateProcess() – создание дочернего процесса. При создании процесса стандартные потоки ввода-вывода перенаправляются: stdin ← pipe1, stdout → файл, stderr → pipe2.

WriteFile() / ReadFile() – запись и чтение данных в/из каналов.

WaitForSingleObject() – ожидание завершения дочернего процесса.

CloseHandle() – закрытие хендлов объектов ОС (файлов, каналов, процессов).

Алгоритм работы программы:

Родительский процесс (parent.exe):

Считывает имя файла из консоли пользователя.

Создаёт файл для записи с помощью CreateFile().

Создаёт два безымянных канала pipe1 и pipe2 с помощью CreatePipe().

Настраивает структуру STARTUPINFO для перенаправления стандартных потоков дочернего процесса.

Запускает дочерний процесс child.exe через CreateProcess().

Считывает строки из консоли и передаёт их в pipe1.

После получения EOF (Ctrl+Z) закрывает pipe1 и считывает ошибки из pipe2, выводя их в консоль.

Ожидает завершения дочернего процесса через WaitForSingleObject().

Дочерний процесс (child.exe):

Считывает строки из stdin (который перенаправлен из pipe1).

Проверяет, начинается ли строка с заглавной буквы (функция isupper()).

Если строка валидна — выводит её в stdout (перенаправлен в файл).

Если строка невалидна — выводит сообщение об ошибке в stderr (перенаправлен в pipe2).

Использованные системные вызовы:

Код программы

parent.c

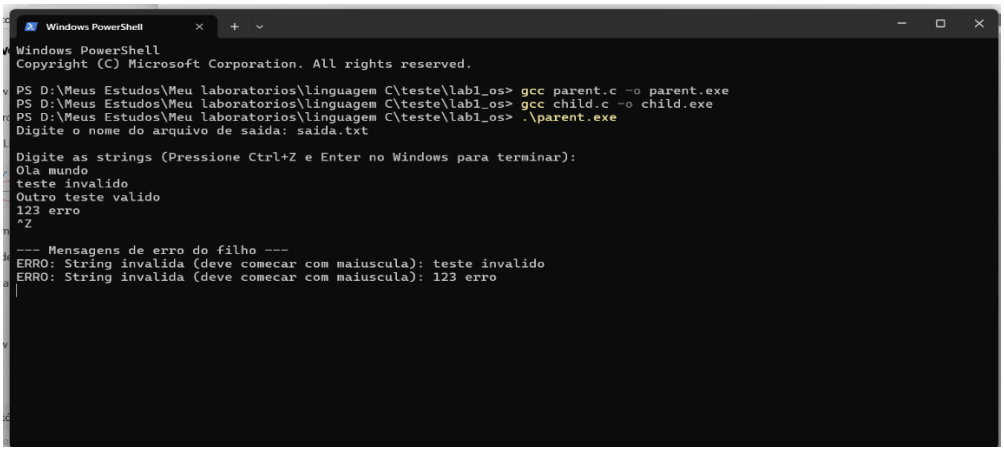
```
#include <windows.h>
#include <stdio.h>
#include <string.h>
int main() {
    char filename[256];
    printf("Digite o nome do arquivo de saida: ");
    if (!fgets(filename, sizeof(filename), stdin)) return 1;
    filename[strcspn(filename, "\n\r")] = 0;
    HANDLE hFile = CreateFileA(filename, GENERIC_WRITE, 0, NULL,
                               CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        printf("Erro ao criar o arquivo!\n");
        return 1;
    }
    HANDLE hRead1, hWrite1, hRead2, hWrite2;
    SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    if (!CreatePipe(&hRead1, &hWrite1, &sa, 0) ||
        !CreatePipe(&hRead2, &hWrite2, &sa, 0)) {
        printf("Erro ao criar os pipes!\n");
        return 1;
    }
    STARTUPINFOA si = {0};
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput  = hRead1;
    si.hStdOutput = hFile;
    si.hStdError  = hWrite2;
```

```
    PROCESS_INFORMATION pi = {0};
    char cmd[] = "child.exe";
    if (!CreateProcessA(NULL, cmd, NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi)) {
        printf("Erro ao criar o processo filho!\n");
        return 1;
    }
    CloseHandle(hRead1);
    CloseHandle(hWrite2);
    CloseHandle(hFile);
    char buffer[4096];
    DWORD bytesWritten, bytesRead;
    printf("\nDigite as strings (Ctrl+Z + Enter para terminar):\n");
    while (fgets(buffer, sizeof(buffer), stdin)) {
        WriteFile(hWrite1, buffer, (DWORD)strlen(buffer), &bytesWritten, NULL);
    }
    CloseHandle(hWrite1);
    printf("\n--- Mensagens de erro do filho ---\n");
    while (ReadFile(hRead2, buffer, sizeof(buffer) - 1, &bytesRead, NULL) && bytesRead > 0) {
        buffer[bytesRead] = '\0';
        printf("%s", buffer);
    }
    CloseHandle(hRead2);
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread)
    printf("---------------------------------\n");
    printf("Processo concluido. Dados validos salvos em '%s'.\n", filename);
    return 0;
}
```

child.c

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char buffer[4096];
    while (fgets(buffer, sizeof(buffer), stdin)) {
        if (strlen(buffer) > 0 && isupper((unsigned char)buffer[0])) {
            printf("%s", buffer);
            fflush(stdout);
        } else {
            fprintf(stderr, "ERRO: String invalida (deve comecar com maiuscula): %s", buffer);
            fflush(stderr);
        }
    }
    return 0;
}
```

Протокол работы программы



Тестирование:

PS D:\Meus Estudos\Meu laboratorios\linguagem C\teste\lab1_os> .\parent.exe

Digite o nome do arquivo de saida: saida.txt

Digite as strings (Pressione Ctrl+Z e Enter no Windows para terminar):

Ola mundo

teste invalido

Outro teste valido

123 erro

^Z

--- Mensagens de erro do filho ---

ERRO: String invalida (deve comecar com maiuscula): teste invalido

ERRO: String invalida (deve comecar com maiuscula): 123 erro

Вывод

В ходе выполнения лабораторной работы были приобретены практические навыки управления процессами в операционной системе Windows и организации межпроцессорного взаимодействия с помощью безымянных каналов (pipes). Была разработана система из двух процессов (parent.exe и child.exe), где родительский процесс создаёт дочерний, перенаправляет его стандартные потоки ввода-вывода в файл и каналы, а затем обменивается с ним данными. Основная трудность заключалась в правильной настройке наследования хендлов через структуру SECURITY_ATTRIBUTES и корректном перенаправлении потоков stdin, stdout и stderr дочернего процесса с помощью структуры STARTUPINFO. Также важно было обеспечить своевременную очистку буферов (fflush) и корректное закрытие неиспользуемых хендлов для предотвращения взаимных блокировок (deadlock) при работе с каналами.